

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf\_085a5118-2cb\agent-a07281b099499f9fa.jsonl`
- SHA-256: `3d1ecb369f0928c590f089113977062c4b3c2f0a6f196ebc2c3bb614522c6825`
- Source modified: `2026-06-20T16:57:09+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf\_085a5118-2cb`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`

Transcript

1. user (2026-06-20T16:56:24.754Z)

Map EVERY per-video artifact write site in the DETECTOR RUNNERS. Read backend/pipeline/detectors/wasb_runner.py, native_wasb_runner.py, tracknet_runner.py, stub_runner.py, wasb_infer.py, base.py. Find where trajectory CSVs, decoded PNG frame caches, and any scratch dirs are written; the {stem}__{vid12}_wasb.csv naming; the WASB_SCRATCH/RALLY_SCRATCH/TMPDIR scratch resolution; keep_frames cleanup. Record exact file:line, path pattern, hardcoded?, keyed_by, and the relevant env/config knobs.

2. assistant (2026-06-20T16:56:26.912Z)

I'll conduct a thorough analysis of the detector runners to map all per-video artifact write sites. Let me start by exploring the structure and then reading each file systematically.

3. user (2026-06-20T16:56:28.056Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/audio/hit_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/audio_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/base.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/fusion_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/person_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/rally_gate.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/tracknet_runner.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_infer.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_detector_keying.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_hit_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_person_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_tracknet_runner.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_wasb_runner.py
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-  
execpolicy/backend/pipeline/audio/hit_detector.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-  
execpolicy/backend/pipeline/detectors/audio_features.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-  
execpolicy/backend/pipeline/detectors/base.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-  
execpolicy/backend/pipeline/detectors/fusion_features.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-  
execpolicy/backend/pipeline/detectors/person_detector.py
```

4. assistant (2026-06-20T16:56:29.926Z)

Now let me find the files in the main project directory:

5. user (2026-06-20T16:56:31.077Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/audio_features.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/base.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/court_gate.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer/backend/pipeline/detectors/fusion_features.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer/backend/pipeline/detectors/native_wasb_runner.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer/backend/pipeline/detectors/person_detector.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/rally_gate.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/stub_runner.py  
C:\Users\avidu\Projects\badminton-highlight-  
indexer/backend/pipeline/detectors/tracknet_runner.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py  
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/detectors/__init__.py
```

6. assistant (2026-06-20T16:56:32.781Z)

Good. Now let me read all the required files:

7. user (2026-06-20T16:56:33.011Z)

```
1      """Abstract base for detector runners (TrackNet, WASB, future native, etc.).  
2  
3      This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1  
4      of the post-observability re-prioritization) to de-risk the WSL/conda dependency for  
5      the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.  
6  
7      The ABC lets us:  
8      - Keep the current WSL adapters working unchanged for Windows users.  
9      - Swap in (or A/B test) a Linux-native implementation later with zero changes to  
10     the hybrid segmenters or the model-agnostic windowing brain.  
11     - Provide a single place to evolve the healthcheck / error contract and thread  
12     detector readiness into per-video reports.  
13  
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):  
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]  
16       Run detector, return the Windows-side path to the produced trajectory CSV  
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.  
18       The caller (hybrid) is responsible for output dir; impls may stage the video  
19       into WSL fs for perf.  
20  
21     - healthcheck() -> tuple[bool, str]  
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.  
23       Message is logged (and can be surfaced to reports or UX in future).  
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
```

```

25
26 Existing behavior is 100% preserved. The only additions are the inheritance and
27 the explicit common type for future implementers.
28
29 How to add a new runner (for docs / future contributors):
30 1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31 2. Implement run_predict and healthcheck (return (bool, str) tuple).
32 3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33    TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34    deliberately static and model-agnostic). If the new detector needs different
35    windowing, it can implement its own or we generalize later.
36 4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37    registry never pulls torch/tf on a machine that doesn't have them).
38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39    DetectorRunner instances (they already do the right thing via duck-typing + the
40    type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42    returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44    windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 """
68
69 import logging
70 import shlex
71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82     treats `~/` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default `~/clips`
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     `~/` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~/":
89         return path

```

```

90     if path.startswith("~/"):
91         return "~/" + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100    and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101    ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102    """
103
104    cfg: Any
105
106    def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107        full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108        argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109        logger.debug("WSL exec: %s", full)
110        return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113    # ---- cache hygiene helpers (shared by both WSL runners) -----
114    def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115        """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117        Returns None if it can't be determined (never raises). Used as a pre-run
118        disk guard so a long extraction doesn't fill the disk mid-flight.
119        """
120        try:
121            res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122            'NR==2{{print $4}}'")
123            except (subprocess.TimeoutExpired, OSError):
124                return None
125            try:
126                return int((res.stdout or "").strip()) / (1024 ** 3)
127            except (ValueError, AttributeError):
128                return None
129
130    def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131        """Best-effort recursive delete of WSL paths (post-success cleanup; never
132        raises)."""
133        paths = [p for p in wsl_paths if p]
134        if not paths:
135            return
136        quoted = " ".join(wsl_tilde_quote(p) for p in paths)
137        try:
138            self._wsl_bash(f"rm -rf {quoted}")
139        except (subprocess.TimeoutExpired, OSError) as e:
140            logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
141
142    def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
143        """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
144
145        Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
146        filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
147        fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
148        trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
149        A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
150        any
151        frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
152        ``cp`` still gets its chance (we never block a genuinely-readable source).
153        """
154        try:

```

```

152         res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
153     except (subprocess.TimeoutExpired, OSError):
154         return None
155     if "READABLE" in (res.stdout or ""):
156         return None
157     return (
158         f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
159         f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
160         f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
161     )
162
163
164 class DetectorRunner(ABC):
165     """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167     @abstractmethod
168     def run_predict(self, video_win_path: str, output_win_dir: str,
169                   video_id: Optional[str] = None) -> Optional[str]:
170         """Run inference and return path to trajectory CSV (or None on failure).
171
172         ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173         videos that happen to share a filename cannot reuse each other's frames or CSV.
174         It is optional for back-compat; callers in the live pipeline always pass it.
175         """
176         pass
177
178     @abstractmethod
179     def healthcheck(self) -> Tuple[bool, str]:
180         """Quick check if the runner (env, weights, etc.) is usable.
181
182         Returns (ok, message). Never raises. Cheap to call before a long run.
183         """
184         pass
185
186     # De facto trajectory contract: the trajectory hybrids call these
187     # unconditionally after run_predict. Concrete runners reuse/re-export the
188     # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189     # a runner with custom windowing supplies its own with the same shapes.
190     parse_trajectory_csv: Callable[..., Any]
191     trajectory_to_action_windows: Callable[..., Any]
192

```

8. user (2026-06-20T16:56:33.219Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
    TrackNetRunner.

```

```

21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45         # extraction that dominates a long clip. Output is bit-identical to the PNG path
46         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47         stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                   f"python -c \"import torch; print('torch', torch.__version__, "
76                   f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)

```

```

79         except FileNotFoundError:
80             return False, "`wsl` not found ? Windows + WSL2 required."
81     except subprocess.TimeoutExpired:
82         return False, "WSL healthcheck timed out."
83     out = (res.stdout or "").strip()
84     if res.returncode != 0:
85         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86     return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117 video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         """
125         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128         # the CSV landed inside WSL while Windows polled the repo-relative path.
129         output_win_dir = os.path.abspath(output_win_dir)
130         os.makedirs(output_win_dir, exist_ok=True)
131         # Normalize for cross-platform basename (tests use Windows paths on Linux).
132         video_win_path = str(video_win_path).replace("\\", "/")
133         base = os.path.basename(video_win_path)
134         stem, ext = os.path.splitext(base)
135         # Unique, content-derived key: same filename + different bytes -> different key.
136         key = f"{stem}__{video_id[:12]}" if video_id else stem
137         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
138         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
139
140         if not self._stage_infer_script():
141             logger.error("Could not stage wasb_infer.py into WSL.")
142             return None
143         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
144         if staged_video is None:

```

```

141         return None
142     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145     # WSL stage filesystem is already low so the user can clean up before it fills.
146     # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149         if free is not None and free < self.cfg.min_free_gb:
150             logger.warning(
151                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                 "may fill the disk ? consider running tools/cleanup_caches.py.",
153                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157     if self.cfg.stream_video:
158         frame_args = "--stream-video "
159     else:
160         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161     cmd = (
162         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163         f"python wasb_infer.py "
164         f"--video {wsl_tilde_quote(staged_video)} "
165         f"{frame_args}"
166         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167         f"--sport {shlex.quote(self.cfg.sport)} "
168         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv'}}")
169
170     logger.info("Running WASB inference on %s ...", base)
171     try:
172         res = self._wsl_bash(cmd)
173     except subprocess.TimeoutExpired:
174         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175         return None
176     if res.returncode != 0:
177         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178         return None
179     if not os.path.exists(expected_csv_win):
180         logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181         return None
182     logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184     # Success ? the trajectory CSV is the durable output; the staged video copy and
185     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187     # so the frame cache + detector cache remain for a resumable re-run.
188     # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189     if not self.cfg.keep_frames:
190         stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191         logger.info("Cache hygiene: removing staged video%s for %s "
192                     "(set indexing.wasb.keep_frames=true to retain).",
193                     "" if self.cfg.stream_video else " + frame cache", key)
194         self._wsl_rm_rf(stale)
195         return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).

```



```

198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

9. user (2026-06-20T16:56:33.431Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10    * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11    ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13    * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14    into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15    * weights / repo / python / scratch come from the environment first
16    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17    ``indexing.wasb.`` config, then a default ? so a box needs no config edits.
18
19    It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__vid12_wasb.csv``
20    name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21    ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22    the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23    (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25    Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
26    (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
27    candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
28    ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
29    run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
30    vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
31    artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
32    (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
33    not the numbers.
34    """
35    import os
36    import shutil
37    import logging
38    import subprocess
39    from dataclasses import dataclass
40    from typing import Any, Dict, List, Optional, Tuple
41
42    from backend.pipeline.detectors.base import DetectorRunner
43    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
44    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
45
46    logger = logging.getLogger(__name__)

```

```

47
48 # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
49 # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
50 _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
51
52
53 @dataclass
54 class NativeWasbConfig:
55     """Resolved settings for a Linux-native WASB run.
56
57     Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
58     over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
59     ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
60     perf win).
61     """
62     repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
63     python_bin: str = "python" # the `wasb` conda env python (invoked by
64     path, no activate)
65     weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
66     indexing.wasb.weights_path)
67     sport: str = "badminton"
68     device: str = "cuda" # cuda | mps | cpu
69     scratch_dir: str = "" # frame-extraction scratch (env); "" = next
70     to the output dir
71     timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
72     timeout)
73     keep_frames: bool = False # retain the decoded frame cache after
74     success (debug only)
75     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
76     clip's
77     # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
78     real GPU
79     # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
80     here.
81     stream_video: bool = True
82
83     @classmethod
84     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
85         w = (idx_cfg or {}).get("wasb", {}) or {}
86         env = os.environ.get
87         # stream_video is tri-state in config: None/absent => native default (stream);
88         an
89         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
90         sv = w.get("stream_video")
91         return cls(
92             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
93             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
94             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
95             sport=w.get("sport", "badminton"),
96             device=env("WASB_DEVICE") or w.get("device", "cuda"),
97             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
98             "",
99             timeout_sec=int(w.get("timeout_sec", 1800)),
100             keep_frames=bool(w.get("keep_frames", False)),
101             stream_video=(True if sv is None else bool(sv)),
102         )
103
104 class NativeWasbRunner(DetectorRunner):
105     """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
106     docstring."""
107
108     def __init__(self, cfg: NativeWasbConfig):
109         self.cfg = cfg

```

```

99
100         # --- low-level native exec (the single place tests patch)
101         -----
102         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
103             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
104             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
105                                   timeout=(self.cfg.timeout_sec or None))
106
107         def _repo_src(self) -> str:
108             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
109
110         def _copy_infer_script(self) -> bool:
111             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
112             configs/."""
113             repo_src = self._repo_src()
114             try:
115                 os.makedirs(repo_src, exist_ok=True)
116                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
117                 return True
118             except OSError as e:
119                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
120                 return False
121
122         def _rm_rf(self, path: Optional[str]) -> None:
123             """Best-effort recursive delete of a native path (post-success cleanup; never
124             raises)."""
125             if path:
126                 shutil.rmtree(path, ignore_errors=True)
127
128         def healthcheck(self) -> Tuple[bool, str]:
129             """Verify weights + repo present and the `wasb` python imports torch (and sees a
130             GPU on cuda)."""
131             if not self.cfg.weights_path:
132                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
133             indexing.wasb.weights_path).")
134             weights = os.path.expanduser(self.cfg.weights_path)
135             if not os.path.exists(weights):
136                 return False, f"WASB weights not found at {weights}"
137             repo_src = self._repo_src()
138             if not os.path.isdir(repo_src):
139                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
140             nttcom/WASB-SBDT "
141                               f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
142             code = "import torch; print('torch', torch.__version__, 'cuda',
143             torch.cuda.is_available())"
144             try:
145                 res = self._run([self.cfg.python_bin, "-c", code])
146             except FileNotFoundError:
147                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
148             WASB_PYTHON)."
149             except subprocess.TimeoutExpired:
150                 return False, "native torch healthcheck timed out."
151             out = (res.stdout or "").strip()
152             if res.returncode != 0:
153                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
154             out).strip()}"
155             if self.cfg.device == "cuda" and "cuda True" not in out:
156                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
157             return True, out
158
159         def run_predict(self, video_win_path: str, output_win_dir: str,
160             video_id: Optional[str] = None) -> Optional[str]:
161             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
162             None.

```

```

153
154     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
155     runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
156     same downstream contract, so the parity comparison is apples-to-apples.
157     """
158     output_dir = os.path.abspath(output_win_dir)
159     os.makedirs(output_dir, exist_ok=True)
160     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
161     norm = str(video_win_path).replace("\\", "/")
162     stem, _ext = os.path.splitext(os.path.basename(norm))
163     # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
164     key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
165     out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
166
167     if not self._copy_infer_script():
168         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
169         return None
170
171     weights = os.path.expanduser(self.cfg.weights_path)
172     argv = [self.cfg.python_bin, "wasb_infer.py",
173            "--video", os.path.abspath(str(video_win_path))]
174     frames_dir: Optional[str] = None
175     if self.cfg.stream_video:
176         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
177         argv.append("--stream-video")
178     else:
179         scratch = self.cfg.scratch_dir or output_dir
180         frames_dir = os.path.join(scratch, f"{key}_frames")
181         argv += ["--frames_out_dir", frames_dir]
182     argv += ["--weights", weights,
183            "--sport", self.cfg.sport,
184            "--device", self.cfg.device,
185            "--out", out_csv]
186
187     logger.info("Running native WASB inference (device=%s) on %s ...",
188               self.cfg.device, os.path.basename(norm))
189     try:
190         res = self._run(argv, cwd=self._repo_src())
191     except FileNotFoundError:
192         logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
193         return None
194     except subprocess.TimeoutExpired:
195         logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
196         return None
197     if res.returncode != 0:
198         logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
199         return None
200     if not os.path.exists(out_csv):
201         logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
202         return None
203     logger.info("WASB trajectory CSV: %s", out_csv)
204
205     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
206     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
207     # (Streaming never materializes frames, so there is nothing to drop on that

```

```

path.)
208         if not self.cfg.keep_frames and frames_dir:
209             logger.info("Cache hygiene: removing frame cache for %s "
210                          "(set indexing.wasb.keep_frames=true to retain).", key)
211             self._rm_rf(frames_dir)
212         return out_csv
213
214         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
215         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
216         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
217

```

10. user (2026-06-20T16:56:34.057Z)

```

1      """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3      A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4      CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
5      (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6      container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7      (``use_gpu=False``); this closes the last GPU/WSL-only gap so the *entire* pipeline is
8      runnable ? and regression-testable ? without a GPU.
9
10     Two modes:
11     * **replay** ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12       pipeline with a committed/golden trajectory; deterministic + cross-OS).
13     * **synth** (default) ? generate a deterministic synthetic trajectory of
14       ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15       gaps, so windowing yields a stable, known number of candidate windows.
16
17     This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18     "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19     (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20     ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21     WSL runners stay the default, so production behaviour is unchanged.
22     """
23     import os
24     import csv as _csv
25     import shutil
26     import logging
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.detectors.base import DetectorRunner
31     # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     @dataclass
38     class StubConfig:
39         """Settings for the CPU stub runner (built from ``config.json`` ->
``indexing.stub``)."""
40         fps: float = 30.0
41         frame_width: float = 1280.0
42         num_rallies: int = 2
43         rally_seconds: float = 4.0
44         gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
windows don't merge)
45         lead_seconds: float = 2.0        # dead lead-in
46         step_px: float = 12.0            # per-frame shuttle displacement during a rally
(>> velocity_thresh)

```

```

47         replay_csv: Optional[str] = None    # if set + exists, emit this CSV verbatim instead
of synthesising
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
51         s = (idx_cfg or {}).get("stub", {}) or {}
52         return cls(
53             fps=float(s.get("fps", 30.0)),
54             frame_width=float(s.get("frame_width", 1280.0)),
55             num_rallies=int(s.get("num_rallies", 2)),
56             rally_seconds=float(s.get("rally_seconds", 4.0)),
57             gap_seconds=float(s.get("gap_seconds", 3.0)),
58             lead_seconds=float(s.get("lead_seconds", 2.0)),
59             step_px=float(s.get("step_px", 12.0)),
60             replay_csv=s.get("replay_csv"),
61         )
62
63
64     class StubDetectorRunner(DetectorRunner):
65         """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
66
67         def __init__(self, cfg: Optional[StubConfig] = None):
68             self.cfg = cfg or StubConfig()
69
70         def healthcheck(self) -> Tuple[bool, str]:
71             return True, "stub CPU detector (no GPU/WSL)"
72
73         def _rows(self) -> List[Tuple[int, int, float, float]]:
74             """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
bursts
75             separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
?
76             well above the default 0.02 velocity threshold ? so every rally frame is
"active"
77             and each burst becomes exactly one candidate window."""
78             cfg = self.cfg
79             rows: List[Tuple[int, int, float, float]] = []
80             frame = 0
81             y = cfg.frame_width / 4.0
82             base_x = cfg.frame_width / 2.0
83             for _ in range(int(cfg.lead_seconds * cfg.fps)):
84                 rows.append((frame, 0, 0.0, 0.0))
85                 frame += 1
86             for r in range(cfg.num_rallies):
87                 for i in range(int(cfg.rally_seconds * cfg.fps)):
88                     x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
89                     rows.append((frame, 1, x, y))
90                     frame += 1
91                 if r != cfg.num_rallies - 1:
92                     for _ in range(int(cfg.gap_seconds * cfg.fps)):
93                         rows.append((frame, 0, 0.0, 0.0))
94                         frame += 1
95             return rows
96
97         def run_predict(self, video_win_path: str, output_win_dir: str,
98                         video_id: Optional[str] = None) -> Optional[str]:
99             """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
100             os.makedirs(output_win_dir, exist_ok=True)
101             stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
"/")))[0]
102             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
103             out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
104
105             if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):

```

```

106         shutil.copyfile(self.cfg.replay_csv, out_csv)
107         logger.info("Stub runner: replayed trajectory %s -> %s",
self.cfg.replay_csv, out_csv)
108         return out_csv
109
110         rows = self._rows()
111         with open(out_csv, "w", newline="") as f:
112             w = _csv.writer(f)
113             w.writerow(["Frame", "Visibility", "X", "Y"])
114             for frame, vis, x, y in rows:
115                 w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
116         logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
out_csv)
117         return out_csv
118
119         # Model-agnostic CSV parsing + windowing ? identical to the real runners.
120         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
121         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
122
123
124         def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
125                                         frame_width: float = 1280.0, tail_pad: float = 2.0,
126                                         step_px: float = 12.0) -> str:
127             """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
128             in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
129             cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
130
131             This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
132             no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
133             learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
134             """
135             rallies: List[Tuple[float, float]] = []
136             with open(labels_csv, newline="", encoding="utf-8") as f:
137                 for row in _csv.DictReader(f):
138                     try:
139                         rallies.append((float(row["start_time"]), float(row["end_time"])))
140                     except (KeyError, ValueError):
141                         continue
142             duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143             nframes = max(1, int(duration * fps))
144             base_x = frame_width / 2.0
145             y = frame_width / 4.0
146             os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147             with open(out_csv, "w", newline="", encoding="utf-8") as f:
148                 w = _csv.writer(f)
149                 w.writerow(["Frame", "Visibility", "X", "Y"])
150                 for i in range(nframes):
151                     t = i / fps
152                     if any(s <= t < e for s, e in rallies):
153                         x = base_x + (step_px if i % 2 else -step_px)
154                         w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
155                     else:
156                         w.writerow([i, 0, "0.000", "0.000"])
157             return out_csv
158

```

11. user (2026-06-20T16:56:34.227Z)

```

1         """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).

```

```

2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = "" # WSL path to the .h5/.keras weights
38         (REQUIRED) queue_length: int = 5
39         stage_in_wsl: bool = True # copy video into WSL fs first
40         (perf) wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
41         in WSL timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False # retain staged video after success
43         (debug only) min_free_gb: float = 0.0 # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),

```



```

59         min_free_gb=float(tn.get("min_free_gb", 0.0)),
60     )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "`wsl` not found ? is this running on Windows with WSL2 installed?"
112             except subprocess.TimeoutExpired:
113                 return False, "WSL healthcheck timed out."
114             out = (res.stdout or "").strip()
115             if res.returncode != 0:
116                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117             return True, out
118
119     # ----- inference
120
121     def run_predict(self, video_win_path: str, output_win_dir: str,
122                    video_id: Optional[str] = None) -> Optional[str]:

```

```

122         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
123
124         ``output_win_dir`` is a Windows directory (under the repo's output/) that
125         WSL writes into via its /mnt view, so the Windows process can read the CSV
126         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
127         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128         a filename cannot collide on the output CSV.
129         """
130         if not self.cfg.weights_path:
131             logger.error(
132                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133                 "in config.json to the WSL path of your trained TrackNetV4 weights."
134             )
135             return None
136
137         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138         # relative paths through unchanged, so WSL would resolve them against the
139         # predict.py cwd instead of the repo (same bug class as wasb_runner).
140         output_win_dir = os.path.abspath(output_win_dir)
141         os.makedirs(output_win_dir, exist_ok=True)
142         # Normalize separators so basename/splitext work when tests (or collector)
143         # pass Windows-style paths on a Linux host.
144         video_win_path = str(video_win_path).replace("\\", "/")
145         base = os.path.basename(video_win_path)
146         stem, ext = os.path.splitext(base)
147
148         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149         if ext.lower() not in (".mp4", ".avi"):
150             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151             return None
152
153         wsl_out = to_wsl_mnt_path(output_win_dir)
154         # Namespace by video_id so two same-filename videos don't collide on the CSV.
155         # predict.py derives its output name from the (staged) video stem, so staging
156         # under the namespaced name propagates into "<key>_predict.csv".
157         key = f"{stem}__{video_id[:12]}" if video_id else stem
158         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160         # Resolve the video path WSL will read.
161         if self.cfg.stage_in_wsl:
162             staged = self._stage_video(video_win_path, f"{key}{ext}")
163             if staged is None:
164                 return None
165             wsl_video = staged
166         else:
167             # No staging: predict.py reads /mnt directly and writes
168             # We cannot rename its output, so fall back to the un-namespaced name.
169             wsl_video = to_wsl_mnt_path(video_win_path)
170             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
171
172         if self.cfg.min_free_gb > 0:
173             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174             if free is not None and free < self.cfg.min_free_gb:
175                 logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                               "consider running tools/cleanup_caches.py.",
177                               free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "

```

```

185         f"--output_dir {wsl_tilde_quote(wsl_out)} "
186         f"--queue_length {self.cfg.queue_length}"
187     )
188     logger.info("Running TrackNet inference on %s ...", base)
189     try:
190         res = self._wsl_bash(cmd)
191     except subprocess.TimeoutExpired:
192         logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193         return None
194
195     if res.returncode != 0:
196         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197         return None
198
199     if not os.path.exists(expected_csv_win):
200         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201         return None
202     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205     # regenerable intermediate). On earlier failure we return above without
cleaning.
206     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207         logger.info("Cache hygiene: removing staged video for %s "
208                     "(set indexing.tracknet.keep_frames=true to retain).", key)
209         self._wsl_rm_rf([wsl_video])
210     return expected_csv_win
211
212 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215     Returns the WSL path to the staged copy, or None on failure.
216     """
217     src_mnt = to_wsl_mnt_path(video_win_path)
218     reason = self.wsl_source_unreadable_reason(src_mnt)
219     if reason:
220         logger.error("Cannot stage video into WSL: %s", reason)
221         return None
222     dest = f"{self.cfg.wsl_stage_dir}/{base}"
223     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224     try:
225         res = self._wsl_bash(cmd)
226     except subprocess.TimeoutExpired:
227         logger.error("Staging video into WSL timed out.")
228         return None
229     if res.returncode != 0 or "OK" not in (res.stdout or ""):
230         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231         return None
232     return dest
233
234 # ----- CSV -> windows
235
236 @staticmethod
237 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239     points: List[TrajectoryPoint] = []
240     with open(csv_win_path, newline="") as f:
241         reader = csv.DictReader(f)
242         for row in reader:
243             try:

```

```

244         points.append(TrajectoryPoint(
245             frame=int(row["Frame"]),
246             visible=int(row["Visibility"]) == 1,
247             x=float(row["X"]),
248             y=float(row["Y"]),
249         ))
250     except (KeyError, ValueError):
251         continue
252     points.sort(key=lambda p: p.frame)
253     return points
254
255 @staticmethod
256 def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                    velocity_thresh: float) -> List[Tuple[int, float]]:
258     """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260     Computes each visible point's normalised velocity from the last *visible* point;
261     an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262     ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263     the caller so identical floats feed the velocity math."""
264     active = [] # (frame_idx, velocity)
265     prev: Optional[TrajectoryPoint] = None
266     for p in points:
267         if not p.visible:
268             prev = None # break the trajectory; absent shuttle = not in flight
269             continue
270         if prev is not None:
271             dframes = p.frame - prev.frame
272             if dframes > 0:
273                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                 velocity = (dist / frame_width) * (fps / dframes)
275                 if velocity > velocity_thresh:
276                     active.append((p.frame, velocity))
277             prev = p
278     return active
279
280 @staticmethod
281 def trajectory_to_action_windows(
282     points: List[TrajectoryPoint],
283     fps: float,
284     frame_width: float,
285     chunk_start: float = 0.0,
286     velocity_thresh: float = 0.02,
287     merge_gap: float = 2.0,
288     min_window_duration: float = 2.0,
289     chunk_index: Optional[int] = None,
290     max_window_duration: float = 0.0,
291     min_split_gap: float = 0.5,
292     window_hard_cap: bool = False,
293     window_inactivity_end: float = 0.0,
294     inactivity_min_density: float = 0.34,
295     inactivity_rally_thresh: float = 0.08,
296     window_blob_fallback: bool = False,
297     window_blob_factor: float = 4.0,
298 ) -> List[Dict[str, Any]]:
299     """Convert a shuttle trajectory into candidate rally windows.
300
301     A frame is "active" when the shuttle is visible AND its inter-frame
302     displacement (normalised by frame width, per second) exceeds
303     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304     ``merge_gap`` seconds of each other are grouped into one window; short
305     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict

```

```

306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310         and a single window swallows several rallies + the dead time between them. When
set, any
311         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315         drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318         over-long window has NO internal inactivity gap to split on (a continuously-
active
319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]

```

```

349         return {
350             "start": group[0][0] / fps + chunk_start,
351             "end": group[-1][0] / fps + chunk_start,
352             "active_frame_count": len(group),
353             "peak_velocity": max(vels),
354             "mean_velocity": sum(vels) / len(vels),
355             "track_id_count": 1, # single shuttle; kept for window-shape parity
356             "chunk_index": chunk_index,
357         }
358
359     groups: List[List[Tuple[int, float]]] = []
360     group = [active[0]]
361     for af in active[1:]:
362         if af[0] - group[-1][0] <= max_gap_frames:
363             group.append(af)
364         else:
365             groups.append(group)
366             group = [af]
367     groups.append(group)
368
369     if max_window_duration > 0:
370         groups = TrackNetRunner._split_overlong_groups(
371             groups, int(max_window_duration * fps), int(min_split_gap * fps),
372             hard_cap=window_hard_cap)
373
374     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379     if window_inactivity_end > 0:
380         tw = int(window_inactivity_end * fps)
381         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388     forced_flags = [False] * len(groups)
389     if window_blob_fallback and max_window_duration > 0:
390         groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391             groups, int(max_window_duration * fps), window_blob_factor)
392
393     windows = []
394     for g, forced in zip(groups, forced_flags):
395         w = _finalize(g)
396         if forced:
397             w["forced_cut"] = True
398         windows.append(w)
399     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401 @staticmethod
402 def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                         min_density: float, rally_thresh: float) -> List[Tuple[int,

```

```

float]]:
404         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
413         if trail_frames <= 0 or len(group) < 2:
414             return group
415         frames = [f for f, _ in group]
416         fast = [1 if v > rally_thresh else 0 for _, v in group]
417         lo = 0
418         fast_sum = 0
419         last_dense = -1
420         for i in range(len(frames)):
421             fast_sum += fast[i]
422             while frames[lo] <= frames[i] - trail_frames:
423                 fast_sum -= fast[lo]
424                 lo += 1
425             win_count = i - lo + 1
426             if win_count > 0 and fast_sum / win_count >= min_density:
427                 last_dense = i
428         return group[: last_dense + 1] if last_dense >= 0 else group
429
430     @staticmethod
431     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
432         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
433         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
434         mid = (group[0][0] + group[-1][0]) / 2.0
435         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
436         return split_i
437
438     @staticmethod
439     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
440                                min_split_frames: int,
441                                hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
442         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
443         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
444         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
445         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
446         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
449         if max_win_frames <= 0:

```

```

450         return groups
451     out: List[List[Tuple[int, float]]] = []
452     work = list(groups)
453     while work:
454         g = work.pop()
455         if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456             out.append(g)
457             continue
458         best_i, best_gap = -1, -1
459         for i in range(1, len(g)):
460             gap = g[i][0] - g[i - 1][0]
461             if gap > best_gap:
462                 best_gap, best_i = gap, i
463         if best_i < 0 or best_gap < min_split_frames:
464             # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465             # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466             if hard_cap:
467                 split_i = TrackNetRunner._midpoint_split_index(g)
468                 work.append(g[:split_i])
469                 work.append(g[split_i:])
470             else:
471                 out.append(g)
472             continue
473             work.append(g[:best_i])
474             work.append(g[best_i:])
475     out.sort(key=lambda grp: grp[0][0])
476     return out
477
478 @staticmethod
479 def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                        blob_factor: float = 4.0
481                        ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482     """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483     inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
484     ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485     signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486     11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487     is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489     degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490     rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491     can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492     that piece came from a forced chop. Iterative worklist to bound stack depth."""
493     if max_win_frames <= 0:
494         return groups, [False] * len(groups)
495     blob_frames = max_win_frames * max(1.0, blob_factor)
496     out: List[List[Tuple[int, float]]] = []
497     flags: List[bool] = []
498     forced = 0
499     for g in groups:
500         if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501             out.append(g)
502             flags.append(False)
503             continue

```



```

504         pieces: List[List[Tuple[int, float]]] = []
505         work = [g]
506         while work:
507             h = work.pop()
508             if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                 pieces.append(h)
510                 continue
511             split_i = TrackNetRunner._midpoint_split_index(h)
512             work.append(h[:split_i])
513             work.append(h[split_i:])
514             pieces.sort(key=lambda grp: grp[0][0])
515             out.extend(pieces)
516             flags.extend([True] * len(pieces))
517             forced += len(pieces) - 1
518         if forced:
519             logger.warning(
520                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522                 "motion), not a bigger cap", forced)
523         return out, flags
524

```

12. user (2026-06-20T16:56:34.573Z)

```

1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]

```

```

38     """
39     import os
40     import os.path as osp
41     import csv
42     import glob
43     import json
44     import argparse
45     import logging
46     from collections import defaultdict
47     from contextlib import contextmanager
48
49     logger = logging.getLogger(__name__)
50
51     CACHE_VERSION = 1
52     _DET_FILE = "det_raw.jsonl"
53     _MANIFEST_FILE = "manifest.json"
54
55
56     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
57         """Compose the WASB eval config via Hydra, overriding model/weights/device.
58
59         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
60         preserved),
61         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
62         ``runner.gpus=[0]``;
63         cpu/mps run with no GPU list so the detector places tensors on ``device``.
64         """
65         from hydra import compose, initialize
66         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
67         with initialize(config_path="configs", version_base=None):
68             cfg = compose(config_name="eval", overrides=[
69                 f"dataset={sport}",
70                 "model=wasb",
71                 f"detector.model_path={weights}",
72                 f"runner.device={device}",
73                 f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to this
74             ])
75         return cfg
76
77     def _frame_id(path: str) -> int:
78         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
79         180)."""
80         stem = osp.splitext(osp.basename(path))[0]
81         digits = "".join(ch for ch in stem if ch.isdigit())
82         return int(digits) if digits else -1
83
84     # ----- #
85     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
86     # ----- #
87     def _cache_paths(cache_dir: str):
88         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
89
90     def default_cache_dir(out_csv: str) -> str:
91         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
92         d = osp.dirname(osp.abspath(out_csv))
93         stem = osp.splitext(osp.basename(out_csv))[0]
94         return osp.join(d, stem + "_wasbcache")
95
96     def _atomic_write_text(path: str, text: str) -> None:
97         """Write then rename so a crash mid-write can't leave a half-written file."""

```

```

99         tmp = path + ".tmp"
100         with open(tmp, "w") as f:
101             f.write(text)
102             f.flush()
103             os.fsync(f.fileno())
104         os.replace(tmp, path)
105
106
107     def _det_plain(det) -> list:
108         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
109         list."""
110         xy = det["xy"]
111         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
112
113     def _dets_to_tracker(plain_list):
114         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
115         array,
116         the tracker does vector math / np.linalg.norm on it)."""
117         import numpy as np
118         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
119                 for p in plain_list]
120
121     def write_manifest(cache_dir: str, manifest: dict) -> None:
122         _, mpath = _cache_paths(cache_dir)
123         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
124
125
126     def read_manifest(cache_dir: str):
127         _, mpath = _cache_paths(cache_dir)
128         if not osp.exists(mpath):
129             return None
130         try:
131             with open(mpath) as f:
132                 return json.load(f)
133         except (json.JSONDecodeError, OSError):
134             return None
135
136
137     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
138                             frames_in: int, frames_total: int, device: str = "cuda") ->
139     bool:
140         """A cache is reusable only if the run that produced it matches this run's inputs.
141         ``device`` defaults to ``cuda`` so caches written before device-keying (no
142         ``device``
143         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
144         cache
145         (detections can differ numerically across devices), and vice-versa.
146         """
147         if not manifest or manifest.get("version") != CACHE_VERSION:
148             return False
149         return (
150             manifest.get("frames_dir") == osp.abspath(frames_dir)
151             and manifest.get("weights") == weights
152             and manifest.get("sport") == sport
153             and manifest.get("frames_in") == frames_in
154             and manifest.get("frames_total") == frames_total
155             and manifest.get("device", "cuda") == device
156         )
157
158     def reset_cache(cache_dir: str) -> None:
159         """Drop any existing cache so the next run starts clean."""

```

```

159     det_jsonl, mpath = _cache_paths(cache_dir)
160     for p in (det_jsonl, mpath):
161         if osp.exists(p):
162             os.remove(p)
163
164
165 def load_and_clean_cache(cache_dir: str):
166     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
167     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
168     prefix so a trailing half-written line from a crash can't corrupt future appends.
169
170     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
171     [x, y, score, scale] candidates (accumulated across every window the frame is in,
172     in window order ? identical to a non-resumed run).
173     """
174     det_jsonl, _ = _cache_paths(cache_dir)
175     det_by_fid = defaultdict(list)
176     valid: list = []
177     if osp.exists(det_jsonl):
178         with open(det_jsonl, "r") as f:
179             for line in f:
180                 s = line.strip()
181                 if not s:
182                     continue
183                 try:
184                     obj = json.loads(s)
185                 except json.JSONDecodeError:
186                     break # truncated trailing line (crash mid-
flush)
187                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
188                     break
189                 valid.append(s)
190                 for fid, plain in obj["f"]:
191                     det_by_fid[int(fid)].extend([list(p) for p in plain])
192                 # Guarantee a clean append boundary by rewriting only the good prefix.
193                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
194     return len(valid), det_by_fid
195
196
197 def _append_windows(fh, objs) -> None:
198     """Append window records as JSONL and durably flush (bounded loss on crash)."""
199     for o in objs:
200         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
201     fh.flush()
202     os.fsync(fh.fileno())
203
204
205 def _atomic_write_trajectory(out_csv: str, rows) -> None:
206     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
207     tmp = out_csv + ".tmp"
208     with open(tmp, "w", newline="") as f:
209         w = csv.writer(f)
210         w.writerow(["Frame", "Visibility", "X", "Y"])
211         w.writerows(rows)
212         f.flush()
213         os.fsync(f.fileno())
214     os.replace(tmp, out_csv)
215
216
217 # ----- #
218 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
219 # ----- #
220 _STREAM_FRAME_FMT = "{:08d}.png"
221
222

```

```

223 def synthetic_frame_path(index: int) -> str:
224     """Stable, index-encoding frame name used by the streaming path.
225
226     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
227     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
228     frames came from disk or from the in-memory stream. The streaming image loader
229     inverts this name back to an index to fetch the decoded frame.
230     """
231     return _STREAM_FRAME_FMT.format(int(index))
232
233
234 def build_windows(frames: list, frames_in: int):
235     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
236     + checkpoint). Pure list math, identical for the disk and streaming paths.
237
238     Returns a list of windows, each a list of ``frames_in`` consecutive paths
239     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
240     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
241     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
242     """
243     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
244
245
246 class SequentialFrameStore:
247     """In-memory sliding buffer over a forward-only frame reader, sized for the
248     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
249
250     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
251     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
252     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
253     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
254     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
255     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
256     earliest index any not-yet-finished window can still ask for) and decode each source
257     frame exactly once via the forward-only ``reader(index)``.
258     """
259
260     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
261         self._reader = reader
262         self._total = int(total)
263         self._window = max(1, int(window))
264         self._buf: dict = {}
265         # On a resumed run the detector's first needed frame is `start_index`; begin
266         # pulling there so the reader can skip (seek past) the already-cached prefix
267         # instead of decoding 0..start_index-1 just to throw them away.
268         self._next = int(start_index) # next index not yet pulled from the reader
269         self._max_seen = int(start_index) - 1
270         self._floor = int(start_index) # lowest index we still keep (never evict
below)
271
272     def get(self, index: int):
273         index = int(index)
274         if index < self._floor:
275             raise KeyError(
276                 f"frame {index} already evicted (below floor {self._floor}); the access
"
277                 f"pattern must stay within a window of the highest frame seen")
278         if index >= self._total:
279             raise KeyError(f"frame {index} out of range (total {self._total})")
280         # Pull forward from the reader until the requested index is buffered.
281         while self._next <= index:
282             self._buf[self._next] = self._reader(self._next)
283             self._next += 1
284         if index > self._max_seen:
285             self._max_seen = index

```

```

286         # Evict frames older than the earliest a still-running window could re-request:
287         # once we've seen index `m`, no future window starts before `m - (window-1)`.
288         new_floor = max(self._floor, self._max_seen - (self._window - 1))
289         for k in range(self._floor, new_floor):
290             self._buf.pop(k, None)
291         self._floor = new_floor
292         return self._buf[index]
293
294
295     def _index_from_synthetic_path(path: str) -> int:
296         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
297
298         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
299         in lockstep with the on-disk naming scheme.
300         """
301         return _frame_id(path)
302
303
304     def _bgr_to_pil_rgb(frame_bgr):
305         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
306         EXACT
307         PIL RGB image WASB's disk path produces.
308
309         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
310         Image.open(PNG).convert('RGB')``;
311         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
312         COLOR_BGR2RGB)``
313         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
314         streamed
315         frame indistinguishable from the on-disk one to everything downstream.
316         """
317         import cv2
318         from PIL import Image
319         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
320
321
322     def _make_streamed_read_image(store, real_read_image):
323         """Build the ``read_image`` replacement used during a streaming detector pass.
324
325         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
326         image,
327         matching ``read_image``'s real return type); any path that doesn't parse to an index
328         falls
329         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
330         testable
331         without the WSL-only ``dataloaders`` package.
332         """
333         def _read_image(path, *args, **kwargs):
334             idx = _index_from_synthetic_path(path)
335             if idx < 0:
336                 return real_read_image(path, *args, **kwargs)
337             return _bgr_to_pil_rgb(store.get(idx))
338         return _read_image
339
340
341     @contextmanager
342     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
343     int = 0):
344         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
345         decoded
346         frames during the detector pass, byte-for-byte as it would over real PNGs.
347
348         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
349         ``cv2.imread``. ``read_image`` (``utils.utils``) does
350         ``Image.open(path).convert('RGB')``,

```

```

341     returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
342     PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
343     byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
344     ``osp.exists`` guard is sidestepped for synthetic stream paths.
345
346     We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
347     calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
348     module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
349     reference).
350
351     ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
352     in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
353     resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
354     original reader is restored on exit.
355     """
356     from dataloaders import dataset_loader as _dl
357     store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
358     real_read_image = _dl.read_image
359     # Rebind read_image in the consuming module for the duration of the detector pass;
360     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
361     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
362     try:
363         yield store
364     finally:
365         _dl.read_image = real_read_image # type: ignore[assignment]
366
367
368     # ----- #
369     # Inference
370     # ----- #
371     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
372            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
373            log_every_batches: int = 50, cache_dir: "str | None" = None,
374            flush_every: int = 1000, fresh: bool = False,
375            frame_loader=None, source_key: "str | None" = None,
376            device: str = "cuda") -> str:
377         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
378
379         Two equivalent ways to supply pixels (output is bit-identical between them):
380
381         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
382
383         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
387         ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
388         to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
389         ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
``num_workers=0``
390         (the in-process shim + buffer can't cross worker processes).

```

```

391
392 ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
393 frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
394 after reboot still matches.
395 """
396 import time
397 import torch
398 from torch.utils.data import DataLoader
399 from detectors import build_detector
400 from trackers import build_tracker
401 from dataloaders import build_img_transforms, build_seq_transforms
402 from dataloaders.dataset_loader import ImageDataset
403 from utils import Center
404
405 streaming = frame_loader is not None
406
407 cfg = _build_cfg(weights, sport, device)
408 frames_in = int(cfg["model"]["frames_in"])
409 input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
410 output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
411
412 if streaming:
413     # `frames` is already the ordered list of (synthetic) frame paths.
414     if not isinstance(frames, (list, tuple)):
415         raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
416         frames = list(frames)
417         frames_dir = source_key or "stream"
418     else:
419         frames_dir = frames
420         frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
421         if limit:
422             frames = frames[:limit]
423         if len(frames) < frames_in:
424             where = frames_dir if not streaming else "video stream"
425             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
426
427     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
428     samples = []
429     for window in build_windows(frames, frames_in):
430         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
431         samples.append({"frames": window, "annos": annos})
432     windows_total = len(samples)
433
434     # ---- cache setup / resume decision ----- #
435     if cache_dir is None:
436         cache_dir = default_cache_dir(out_csv)
437         os.makedirs(cache_dir, exist_ok=True)
438         det_jsonl, _ = _cache_paths(cache_dir)
439         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440         manifest = None if fresh else read_manifest(cache_dir)
441         resume = manifest_compatible(
442             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443             frames_in=frames_in, frames_total=len(frames), device=device,
444         )
445         if resume:
446             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
447             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
448         else:
449             if manifest is not None:
450                 logger.info("cache incompatible with this run -> starting fresh")

```



```

451         reset_cache(cache_dir)
452         windows_done, det_by_fid = 0, defaultdict(list)
453
454     base_manifest = {
455         "version": CACHE_VERSION,
456         "frames_dir": cache_key,
457         "weights": weights,
458         "sport": sport,
459         "frames_in": frames_in,
460         "frames_total": len(frames),
461         "device": device,
462         "windows_total": windows_total,
463         "windows_done": windows_done,
464     }
465     write_manifest(cache_dir, base_manifest)
466
467     # ---- detector pass over the un-cached windows ----- #
468     remaining = samples[windows_done:]
469     if remaining:
470         _, transform_test = build_img_transforms(cfg)
471         try:
472             _, seq_transform_test = build_seq_transforms(cfg)
473         except Exception:
474             seq_transform_test = None
475         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
476                         transform=transform_test, seq_transform=seq_transform_test,
477                         is_train=False)
478         # Streaming forces single-process loading: the in-memory frame store + the
479         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
480         workers.
481         loader_workers = 0 if streaming else num_workers
482         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
483                             num_workers=loader_workers)
484         detector = build_detector(cfg)
485
486         from contextlib import ExitStack
487
488         t0 = time.time()
489         done = windows_done
490         win_idx = windows_done
491         log_every = max(1, log_every_batches)
492         flush_n = max(1, flush_every)
493         pending = []
494         logger.info(f"detecting on {len(remaining)} remaining windows "
495                    f"(total {windows_total}, batch={batch_size},
496                    f"source={'video-stream' if streaming else 'frames-dir'})...")
497         det_f = open(det_jsonl, "a")
498         try:
499             with ExitStack() as stack:
500                 stack.enter_context(torch.no_grad())
501                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
502                 # to the
503                 # in-memory decoded frame for that synthetic path; every other line of
504                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
505                 # exactly.
506                 # The detector's first needed frame is `windows_done` (window i starts
507                 # at
508                 # frame i), so prime the store there for a resume.
509                 if streaming:
510                     stack.enter_context(
511                         _streaming_read_image_patch(frame_loader, len(frames),
512                                                         window=frames_in,
513                                                         start_index=windows_done))
514                 for bi, batch in enumerate(loader):

```

```

508         imgs, _hms, trans = batch[0], batch[1], batch[2]
509         img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
510         batch_results, _ = detector.run_tensor(imgs, trans)
511         nb = imgs.shape[0]
512         for ib in range(nb):
513             frames_payload = []
514             for ie in sorted(batch_results[ib].keys()):
515                 p = img_paths[ie][ib]
516                 fid = _frame_id(p)
517                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
518                 frames_payload.append([fid, plain])
519                 det_by_fid[fid].extend(plain)
520             pending.append({"w": win_idx, "f": frames_payload})
521             win_idx += 1
522         done += nb
523         if len(pending) >= flush_n or done >= windows_total:
524             _append_windows(det_f, pending)
525             pending = []
526             base_manifest["windows_done"] = done
527             write_manifest(cache_dir, base_manifest)
528         if (bi + 1) % log_every == 0 or done >= windows_total:
529             el = time.time() - t0
530             new_done = done - windows_done
531             rate = new_done / el if el > 0 else 0.0
532             eta = (windows_total - done) / rate if rate > 0 else 0.0
533             logger.info(f"{done}/{windows_total} "
534                        f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
535                        f"elapsed {el:.0f}s | ETA {eta:.0f}s")
536         if pending:
537             _append_windows(det_f, pending)
538             base_manifest["windows_done"] = done
539             write_manifest(cache_dir, base_manifest)
540         finally:
541             det_f.close()
542             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
543     else:
544         logger.info("detector cache already complete -> tracker-only re-run")
545
546     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
547     tracker = build_tracker(cfg)
548     tracker.refresh()
549     rows = []
550     for fid in sorted(det_by_fid.keys()):
551         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
552         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
553
554     _atomic_write_trajectory(out_csv, rows)
555     visible = sum(1 for _, v, *_ in rows if v)
556     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
557     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
558     return out_csv
559
560
561 def extract_frames(video_path: str, frames_dir: str) -> str:
562     """Decode a video to PNG frames named by source frame index (00000000.png ...).
563
564     Idempotent: if the dir already holds the expected number of frames (matching the
565     container's reported frame count) we skip re-decoding ? re-extraction after a
566     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
567     extracting with ffmpeg on the host and passing --frames_dir.
568     """
569     import cv2
570     os.makedirs(frames_dir, exist_ok=True)

```

```

571     cap = cv2.VideoCapture(video_path)
572     if not cap.isOpened():
573         raise SystemExit(f"cannot open video: {video_path}")
574     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
575     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
576     if expected > 0 and existing >= expected:
577         cap.release()
578         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
579         return frames_dir
580     i = 0
581     while True:
582         ok, frame = cap.read()
583         if not ok:
584             break
585         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
586         i += 1
587     cap.release()
588     logger.info(f"extracted {i} frames -> {frames_dir}")
589     return frames_dir
590
591
592     def _probe_frame_count(video_path: str) -> int:
593         """Container-reported frame count via cv2 (0 if unknown). Used to size the
594         stream."""
595         import cv2
596         cap = cv2.VideoCapture(video_path)
597         if not cap.isOpened():
598             raise SystemExit(f"cannot open video: {video_path}")
599         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
600         cap.release()
601         return n
602
603     def make_video_frame_loader(video_path: str):
604         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
605
606         ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
607         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
608         detector's sliding-window order); it reads sequentially and only uses a frame-seek
609         to
610         skip forward when a *resume* starts past the current position. Each ``cap.read()``
611         returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
612         would
613         yield (PNG is lossless), so the detector sees identical pixels to the disk path.
614
615         Returns the count hint from the container so the caller can build the frame list;
616         ``release`` closes the capture.
617         """
618         import cv2
619         cap = cv2.VideoCapture(video_path)
620         if not cap.isOpened():
621             raise SystemExit(f"cannot open video: {video_path}")
622         count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
623         state = {"pos": 0} # next index cap.read() will return
624
625         def frame_loader(index: int):
626             index = int(index)
627             if index < state["pos"]:
628                 raise RuntimeError(
629                     f"streaming decode is forward-only; got index {index} after
630                     {state['pos']}")
631             if index > state["pos"]:
632                 # Forward skip only happens when resuming past cached windows. Seek once.
633                 cap.set(cv2.CAP_PROP_POS_FRAMES, index)
634                 state["pos"] = index

```

```

632         ok, frame = cap.read()
633         if not ok:
634             raise RuntimeError(f"failed to decode frame {index} from {video_path}")
635         state["pos"] += 1
636         return frame
637
638     def release():
639         cap.release()
640
641     return frame_loader, count_hint, release
642
643
644     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
645                             limit: int = 0, batch_size: int = 8,
646                             log_every_batches: int = 50, cache_dir: str | None = None,
647                             flush_every: int = 1000, fresh: bool = False,
648                             device: str = "cuda") -> str:
649         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
650         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
651         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
652         ``run``'s docstring for why.
653         """
654         frame_loader, count_hint, release = make_video_frame_loader(video_path)
655         try:
656             if count_hint <= 0:
657                 raise SystemExit(
658                     f"could not determine frame count for {video_path}; cannot stream "
659                     f"(fall back to frame extraction with --frames_dir)")
660             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
661             source_key = "video:" + osp.abspath(video_path)
662             return run(frame_list, weights, out_csv, sport, limit,
663                       batch_size=batch_size, num_workers=0,
664                       log_every_batches=log_every_batches, cache_dir=cache_dir,
665                       flush_every=flush_every, fresh=fresh,
666                       frame_loader=frame_loader, source_key=source_key, device=device)
667         finally:
668             release()
669
670
671     def main():
672         ap = argparse.ArgumentParser()
673         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
674         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
675         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
676         ap.add_argument("--stream-video", action="store_true",
677                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
678                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
679         ap.add_argument("--weights", required=True)
680         ap.add_argument("--out", required=True)
681         ap.add_argument("--sport", default="badminton")
682         ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
683                         help="torch device for the detector (default cuda = the WSL parity
path)")
684         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
685         ap.add_argument("--batch-size", type=int, default=8)
686         ap.add_argument("--num-workers", type=int, default=4)
687         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
688         ap.add_argument("--cache-dir", default=None,

```

```

689             help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
690         ap.add_argument("--flush-every", type=int, default=1000,
691             help="fsync the detector cache every N windows (bounds reboot
loss)")
692         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
693         args = ap.parse_args()
694
695         if args.stream_video:
696             if not args.video:
697                 raise SystemExit("--stream-video requires --video")
698             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
699                 batch_size=args.batch_size,
700                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
701                 flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
702             return
703
704         frames_dir = args.frames_dir
705         if args.video:
706             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707         if not frames_dir:
708             raise SystemExit("provide --frames_dir or --video")
709         run(frames_dir, args.weights, args.out, args.sport, args.limit,
710             batch_size=args.batch_size, num_workers=args.num_workers,
711             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712             flush_every=args.flush_every, fresh=args.fresh, device=args.device)
713
714
715     if __name__ == "__main__":
716         main()
717

```

13. assistant (2026-06-20T16:56:37.058Z)

Now let me search for additional patterns related to scratch directory resolution and cleanup:

14. user (2026-06-20T16:56:38.136Z)

```

backend\pipeline\detectors\native_wasb_runner.py:16:    (`WASB_WEIGHTS` / `WASB_REPO_DIR` /
`WASB_PYTHON` / `WASB_SCRATCH`), then
backend\pipeline\detectors\native_wasb_runner.py:87:        scratch_dir=env("WASB_SCRATCH")
or env("RALLY_SCRATCH") or env("TMPDIR") or "",

```

15. user (2026-06-20T16:56:38.634Z)

```

backend\pipeline\detectors\base.py:129:    def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
backend\pipeline\detectors\wasb_runner.py:41:        keep_frames: bool = False # retain staged
video + frame cache after success (debug only)
backend\pipeline\detectors\wasb_runner.py:62:            keep_frames=bool(w.get("keep_frames",
False)),
backend\pipeline\detectors\wasb_runner.py:189:            if not self.cfg.keep_frames:
backend\pipeline\detectors\wasb_runner.py:192:                "(set
indexing.wasb.keep_frames=true to retain).",
backend\pipeline\detectors\wasb_runner.py:194:                self._wsl_rm_rf(stale)
backend\pipeline\detectors\native_wasb_runner.py:68:        keep_frames: bool = False #
retain the decoded frame cache after success (debug only)
backend\pipeline\detectors\native_wasb_runner.py:89:        keep_frames=bool(w.get("keep_frames", False)),
backend\pipeline\detectors\native_wasb_runner.py:120:    def _rm_rf(self, path: Optional[str])
-> None:

```

```

backend\pipeline\detectors\native_wasb_runner.py:123:         shutil.rmtree(path,
ignore_errors=True)
backend\pipeline\detectors\native_wasb_runner.py:208:         if not self.cfg.keep_frames and
frames_dir:
backend\pipeline\detectors\native_wasb_runner.py:210:             "(set
indexing.wasb.keep_frames=true to retain).", key)
backend\pipeline\detectors\native_wasb_runner.py:211:             self._rm_rf(frames_dir)
backend\pipeline\detectors\tracknet_runner.py:42:         keep_frames: bool = False
# retain staged video after success (debug only)
backend\pipeline\detectors\tracknet_runner.py:58:
keep_frames=bool(tn.get("keep_frames", False)),
backend\pipeline\detectors\tracknet_runner.py:206:         if self.cfg.stage_in_wsl and not
self.cfg.keep_frames:
backend\pipeline\detectors\tracknet_runner.py:208:             "(set
indexing.tracknet.keep_frames=true to retain).", key)
backend\pipeline\detectors\tracknet_runner.py:209:             self._wsl_rm_rf([wsl_video])

```

16. assistant (2026-06-20T16:56:40.862Z)

Perfect. Now let me check for any cache directory patterns in wasb_infer.py:

17. user (2026-06-20T16:56:41.606Z)

```

20-
21- * The detector pass writes its per-window raw outputs incrementally to
22:   ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23:   records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24:   most ``--flush-every`` windows of GPU work instead of the whole pass.
25- * On restart we replay the cached windows, skip the DataLoader ahead to the first
--
29:   not the GPU hour.
30-
31:The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32-to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33-weights, sport, window size, or frame count change.
34-
35-Usage (in WSL, from ~\models\WASB-SBDT\src):
36:   python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37:   --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-every
1000] [--fresh]
38:   """
--
50-
51-CACHE_VERSION = 1
52:_DET_FILE = "det_raw.jsonl"
53:_MANIFEST_FILE = "manifest.json"
54-
55-
--
84-# Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
85-# ----- #
86:def _cache_paths(cache_dir: str):
87:     return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
88-
89-
90:def default_cache_dir(out_csv: str) -> str:
91:     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
92:     d = osp.dirname(osp.abspath(out_csv))
93:     stem = osp.splitext(osp.basename(out_csv))[0]
94:     return osp.join(d, stem + "_wasbcache")
95-
96-
--
119-

```

```

120-
121: def write_manifest(cache_dir: str, manifest: dict) -> None:
122:     _, mpath = _cache_paths(cache_dir)
123:     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
124-
125-
126: def read_manifest(cache_dir: str):
127:     _, mpath = _cache_paths(cache_dir)
128:     if not osp.exists(mpath):
129:         return None
130:     --
131-
132-
133-
134-
135-
136-
137: def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
138:                          frames_in: int, frames_total: int, device: str = "cuda") -> bool:
139:     """A cache is reusable only if the run that produced it matches this run's inputs.
140:     --
141:     """
142:     return False
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157: def reset_cache(cache_dir: str) -> None:
158:     """Drop any existing cache so the next run starts clean."""
159:     det_jsonl, mpath = _cache_paths(cache_dir)
160:     for p in (det_jsonl, mpath):
161:         if osp.exists(p):
162:             osp.remove(p)
163:     --
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-
368-
369-
370-
371-
372-
373-
374-
375-
376-
377-
378-
379-
380-
381-
382-
383-
384-
385-
386-
387-
388-
389-
390-
391-
392-
393-
394-
395-
396-
397-
398-
399-
400-
401-
402-
403-
404-
405-
406-
407-
408-
409-
410-
411-
412-
413-
414-
415-
416-
417-
418-
419-
420-
421-
422-
423-
424-
425-
426-
427-
428-
429-
430-
431-
432-
433-

```

```

434- # ---- cache setup / resume decision ----- #
435- if cache_dir is None:
436-     cache_dir = default_cache_dir(out_csv)
437-     os.makedirs(cache_dir, exist_ok=True)
438-     det_jsonl, _ = _cache_paths(cache_dir)
439-     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440-     manifest = None if fresh else read_manifest(cache_dir)
441-     resume = manifest_compatible(
442-         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443-         frames_in=frames_in, frames_total=len(frames), device=device,
444-     )
445-     if resume:
446-         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
447-         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows already
detected")
448-     else:
449-         if manifest is not None:
450-             logger.info("cache incompatible with this run -> starting fresh")
451-             reset_cache(cache_dir)
452-             windows_done, det_by_fid = 0, defaultdict(list)
453-
454-     base_manifest = {
455-         "version": CACHE_VERSION,
456-         "frames_dir": cache_key,
457-         "weights": weights,
458-         "sport": sport,
459-         --
460-         "windows_done": windows_done,
461-     }
462-     write_manifest(cache_dir, base_manifest)
463-
464- # ---- detector pass over the un-cached windows ----- #
465- --
466-     pending = []
467-     base_manifest["windows_done"] = done
468-     write_manifest(cache_dir, base_manifest)
469-     if (bi + 1) % log_every == 0 or done >= windows_total:
470-         el = time.time() - t0
471-         --
472-         _append_windows(det_f, pending)
473-         base_manifest["windows_done"] = done
474-         write_manifest(cache_dir, base_manifest)
475-     finally:
476-         det_f.close()
477-     --
478-
479-
480-
481-
482-
483-
484-
485-
486-
487-
488-
489-
490-
491-
492-
493-
494-
495-
496-
497-
498-
499-
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518-
519-
520-
521-
522-
523-
524-
525-
526-
527-
528-
529-
530-
531-
532-
533-
534-
535-
536-
537-
538-
539-
540-
541-
542-
543-
544-
545-
546-
547-
548-
549-
550-
551-
552-
553-
554-
555-
556-
557-
558-
559-
560-
561-
562-
563-
564-
565-
566-
567-
568-
569-
570-
571-
572-
573-
574-
575-
576-
577-
578-
579-
580-
581-
582-
583-
584-
585-
586-
587-
588-
589-
590-
591-
592-
593-
594-
595-
596-
597-
598-
599-
600-
601-
602-
603-
604-
605-
606-
607-
608-
609-
610-
611-
612-
613-
614-
615-
616-
617-
618-
619-
620-
621-
622-
623-
624-
625-
626-
627-
628-
629-
630-
631-
632-
633-
634-
635-
636-
637-
638-
639-
640-
641-
642-
643-
644-
645-
646-
647-
648-
649-
650-
651-
652-
653-
654-
655-
656-
657-
658-
659-
660-
661-
662-
663-
664-
665-
666-
667-
668-
669-
670-
671-
672-
673-
674-
675-
676-
677-
678-
679-
680-
681-
682-
683-
684-
685-
686-
687-
688-
689-
690-
691-
692-
693-
694-
695-
696-
697-
698-
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-

```



```

579:         return frames_dir
580-     i = 0
581-     while True:
582-         --
583-         if not ok:
584-             break
585:         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
586-         i += 1
587-     cap.release()
588:     logger.info(f"extracted {i} frames -> {frames_dir}")
589-     return frames_dir
590-
591-
592-
644-def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
645-                           limit: int = 0, batch_size: int = 8,
646-                           log_every_batches: int = 50, cache_dir: "str | None" = None,
647-                           flush_every: int = 1000, fresh: bool = False,
648-                           device: str = "cuda") -> str:
649-
650-     --
651-     raise SystemExit(
652-         f"could not determine frame count for {video_path}; cannot stream "
653-         f"(fall back to frame extraction with --frames_dir)")
654-     frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
655-     source_key = "video:" + osp.abspath(video_path)
656-     return run(frame_list, weights, out_csv, sport, limit,
657-                batch_size=batch_size, num_workers=0,
658-                log_every_batches=log_every_batches, cache_dir=cache_dir,
659-                flush_every=flush_every, fresh=fresh,
660-                frame_loader=frame_loader, source_key=source_key, device=device)
661-
662-
663-
671-def main():
672-     ap = argparse.ArgumentParser()
673:     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
674:     ap.add_argument("--video", help="video file; frames are extracted internally (in WSL)")
675:     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is used")
676-
677-     --
678-     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
679-     ap.add_argument("--cache-dir", default=None,
680:                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
681-     ap.add_argument("--flush-every", type=int, default=1000,
682:                     help="fsync the detector cache every N windows (bounds reboot loss)")
683-
684-     --
685-     run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
686-                          batch_size=args.batch_size,
687-                          log_every_batches=args.log_every_batches,
688-                          cache_dir=args.cache_dir,
689-                          flush_every=args.flush_every, fresh=args.fresh,
690-                          device=args.device)
691-     return
692-
693-
694-
704:     frames_dir = args.frames_dir
705-     if args.video:
706:         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707-     if not frames_dir:
708-         raise SystemExit("provide --frames_dir or --video")
709-     run(frames_dir, args.weights, args.out, args.sport, args.limit,
710-         batch_size=args.batch_size, num_workers=args.num_workers,
711-         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712-         flush_every=args.flush_every, fresh=args.fresh, device=args.device)
713-

```

18. assistant (2026-06-20T16:56:43.831Z)

Now I have all the information I need. Let me compile a comprehensive structured output with all the artifact write sites.

19. user (2026-06-20T16:57:02.731Z)

Structured output provided successfully

20. assistant (2026-06-20T16:57:09.007Z)

Perfect! I've completed a comprehensive mapping of all per-video artifact write sites in the detector runners. Here's a summary of my findings:

Summary

I've mapped **every per-video artifact write site** across all four detector runners in the badminton-highlight-indexer:

Key Artifacts Identified:

- Trajectory CSVs** - The primary output:
 - WASB (WSL & native): `{stem}__{vid12}_wasb.csv``
 - TrackNet (WSL): `{stem}__{vid12}_predict.csv``
 - Stub (CPU): `{stem}__{vid12}_stub.csv``
- Frame Caches** - Optional PNG extraction directories:
 - Path: `{wsl_stage_dir or scratch_dir}/{key}_frames/``
 - Keyed by `video_id`` (file MD5) to prevent filename collisions
 - Cleaned on success unless `keep_frames=true`` (debug-only override)
- Detector Cache** - Resumable WASB detection state:
 - Location: `{output_dir}/{key}_wasb_wasbcache/``
 - Contains `det_raw.jsonl`` (incremental detections) and `manifest.json`` (metadata)
 - Survives reboots; invalidated if inputs change
- Staged Videos** - Intermediate copies in WSL:
 - Path: `{wsl_stage_dir}/{key}{ext}`` (default `~/clips``)
 - Cleaned on success unless `keep_frames=true``

Scratch Directory Resolution (native_wasb_runner):

- `WASB_SCRATCH`` > `RALLY_SCRATCH`` > `TMPDIR`` > `output_dir``
- Used only when NOT streaming video frames

Config & Environment Knobs:

All relevant settings controlling artifact locations and cleanup behavior are documented in the output, including WSL stage directories, `keep_frames` toggles, device selection, and environment variable overrides for the native runner.